

Fintech Phase 1 - Detailed Project Description

1. Executive Summary

Fintech Phase 1 is a comprehensive, AI-powered financial technology platform designed to offer institutional-level analytics, data processing, and predictive modeling for retail investors. The system integrates real-time market data with advanced machine learning models to facilitate data-driven investment decisions.

2. Global Architecture

The platform operates on a robust, scalable microservices architecture powered by Docker Compose. The environment consists of 5 tightly integrated containers:

- **Frontend (UI Node):** Built with Next.js (React), running on port 3000. It serves as the main interactive user interface.
- **Backend (Core API Node):** A FastAPI (Python) service on port 8000. It handles all business logic, user management, portfolio tracking, paper trading, and database communications.
- **ML Service (Analytics Node):** A dedicated FastAPI (Python) service on port 8001 that manages computationally heavy workflows, machine learning model training, and AI inference.
- **Database:** PostgreSQL extended with TimescaleDB (port 5432) for highly optimized time-series data storage, essential for fast querying of historical stock prices.
- **Cache & Message Broker:** Redis (port 6379) for high-speed caching of frequent requests and asynchronous task queueing for background ML jobs.

3. Detailed Sub-Systems and Modules

3.1 Frontend Service (Next.js Application)

The frontend is highly modularized via an internal dashboard router:

- **Dashboard:** The central command center aggregating portfolio balance, daily P&L, market movers, and active alerts.
- **Authentication:** Handles user onboarding, secure login, JWT management, and session recovery.
- **Technical Analysis:** Interactive charting with indicators like Moving Averages, RSI, MACD, and Bollinger Bands.
- **Fundamental Analysis:** Company health metrics, balance sheets, and AI-summarized health scores.
- **ML Predictions (Risk & Strategies):** Visualizes ML outputs. Shows Value at Risk (VaR), portfolio beta, Sharpe ratios, and algorithmic trading strategies.
- **Backtesting Simulator:** Lets users define trading parameters, select timelines, and simulate historical returns.
- **Trading Module:** Facilitates paper (simulated) trading with Market, Limit, and Stop orders.
- **Community & News:** A social feed for sharing trades and a real-time news aggregator with NLP sentiment scores.
- **Education:** A learning management system offering courses on financial topics.

3.2 Backend Service (Core API)

Handles standard operations, CRUD, and state management via structured API endpoints:

- **Data Models:** Manages entities like User, Stock (master data), Portfolio & Order (balances, active positions, trades), Alert (user-configured price triggers), Education/Social, and Strategy (trading rules).
- **Authentication (/auth & /users):** JWT generation, profiles, and settings.
- **Market Data (/stocks):** Fetching available equities, standard historical data, and metadata.
- **Trading (/trading & /alerts):** Submitting/canceling paper trades and managing user alerts.
- **Community Features (/social & /education):** Endpoints for social feeds and educational progress.

3.3 Machine Learning Service (ML Analytics)

This dedicated service runs on a separate Python environment from the Core API to offload heavy computations:

- **Training Service:** Periodically retrains predictive models (e.g., LSTM/Transformers) on updated historical data.
- **Prediction Service:** Outputs stock price forecasts for the next N days with confidence intervals.
- **Risk Service:** Evaluates maximum drawdown, standard metrics (Alpha, Beta), and runs Monte Carlo simulations for portfolio risks.
- **Backtest Service:** Replays historical data against user-defined technical rules to simulate P&L and win rates.
- **Sentiment Service:** Uses NLP to process financial news and daily headlines, assigning bullish/bearish scores to assets.
- **Feature & Fundamental Service:** Prepares raw historical data by calculating rolling indicators and normalizing fundamental ratios for ML models.

4. Folder Structure and Technical Stack

- `/frontend/` : Contains the Next.js frontend code (React, Tailwind CSS/PostCSS, TypeScript configuration).
- `/backend/` : Contains the FastAPI backend code (`app` folder with API routes and models, `alembic` for database migrations via SQLAlchemy, Python dependencies in `pyproject.toml`).
- `/ml-service/` : Contains the FastAPI ML analytics service, data models, and scripts for training/predictions.
- `docker-compose.yml` : Orchestrates the 5 services, their internal `fintech_network` , and volume persistence for PostgreSQL and Redis.

5. Security and Future Roadmap

- **Security:** Isolated docker networks, JWT token structures, and secure PostgreSQL bindings.
- **Future Enhancements:**
 - Implementation of WebSockets for sub-millisecond real-time market data updates.
 - Live broker integration (e.g., Alpaca) for transitioning from paper to real capital trading.
 - Kubernetes (K8s) orchestration for autoscaling the ML models during high-market-volume hours.